

Guía de Integración y Cableado de Arquitectura (Panel Prem Vida)

Este documento contiene la especificación de ingeniería para el cableado definitivo del panel administrativo, uniendo la interfaz visual *glassmorphic* de Stitch con las funciones transaccionales y de tiempo real de Antigravity 2.0 soportadas por Supabase.

1. Inicialización del Módulo Core

La integración requiere la inicialización segura del cliente de Supabase mediante variables de entorno persistidas en el almacenamiento local del navegador (localStorage), garantizando la portabilidad del panel sin exponer credenciales en reposo en el repositorio de código estático.

2. Estructura del Script de Inicialización Integrado

A continuación se detalla el bloque de código de inicialización que debe inyectarse en el archivo HTML principal (code.html) dentro del tag de tipo módulo (<script type="module">) para dar vida e interactividad en tiempo real al inventario de productos veganos.

```
import {
  fetchProducts,
  insertProduct,
  uploadProductImage,
  processXeroInventoryCSV,
  fetchExpenses,
  fetchSupplierOrders
} from './js/core/logic.js';

let supabaseClient = null;

// 1. Inicialización y Validación de Credenciales
function initializeApp() {
  const url = localStorage.getItem('SUPABASE_URL');
  const key = localStorage.getItem('SUPABASE_ANON_KEY');

  if (!url || !key) {
    openSettingsModal();
    return;
  }

  // Instanciación global del cliente SDK de Supabase
  supabaseClient = window.supabase.createClient(url, key);

  // Carga inicial del pipeline de datos
  loadDashboardData();
}
```

```

    // Activación del canal WebSocket en tiempo real
    setupRealtimeSynchronization();
}

// 2. Orquestación de Sincronización en Tiempo Real (WebSockets)
function setupRealtimeSynchronization() {
    if (!supabaseClient) return;

    const productChannel = supabaseClient
        .channel('public:products')
        .on('postgres_changes', { event: '*', schema: 'public', table:
'products' }, (payload) => {
            console.log('Cambio detectado en productos:', payload);
            // Recarga reactiva de los elementos en pantalla sin refrescar
la página
            refreshProductTable();
        })
        .subscribe();
}

// 3. Manejo de Eventos y Registro de Formularios
document.getElementById('product-form')?.addEventListener('submit',
async (e) => {
    e.preventDefault();
    if (!supabaseClient) return;

    const submitBtn = e.target.querySelector('button[type="submit"]');
    submitBtn.disabled = true;
    submitBtn.innerText = 'Guardando...';

    try {
        const fileInput = document.getElementById('product-image-file');
        const file = fileInput?.files[0];
        let imageUrl = '';

        if (file) {
            imageUrl = await uploadProductImage(supabaseClient, file);
        }

        const productData = {
            sku: document.getElementById('product-sku').value,
            name: document.getElementById('product-name').value,
            category: document.getElementById('product-category').value,
            price:
parseFloat(document.getElementById('product-price').value),
            stock: parseInt(document.getElementById('product-stock').value),
            image_url: imageUrl

```

```

    };

    await insertProduct(supabaseClient, productData);
    closeNewProductModal();
    e.target.reset();
  } catch (error) {
    console.error('Error en el flujo de inserción:', error);
    alert('Error al registrar el producto.');
```

```

  } finally {
    submitBtn.disabled = false;
    submitBtn.innerText = 'Guardar Producto';
  }
});

// 4. Conexión del Pipeline de Importación de XERO
document.getElementById('xero-csv-input')?.addEventListener('change',
async (e) => {
  if (!supabaseClient) return;
  const file = e.target.files[0];
  if (!file) return;

  const reader = new FileReader();
  reader.onload = async (evt) => {
    try {
      const csvText = evt.target.result;
      await processXeroInventoryCSV(supabaseClient, csvText);
      alert('Inventario de XERO importado con éxito por coincidencia
difusa.');
```

```

    } catch (err) {
      console.error('Fallo en la importación masiva:', err);
      alert('Error al procesar el archivo de XERO.');
```

```

    }
  };
  reader.readAsText(file);
});

// Arrancar la aplicación tras la carga del DOM
document.addEventListener('DOMContentLoaded', initializeApp);
```

3. Métricas y Renderizado Reactivo (Bento Grid)

Una vez conectado el cliente, las funciones de cálculo asíncronas iteran sobre la data recuperada para computar en memoria las métricas del panel administrativo expuestas en la moneda local (**Bs.**):

- **Valuación Total:** Sumatoria del producto cruzado entre el precio unitario y el stock de cada ítem de inventario.

- **Stock Crítico:** Filtrado de productos con inventario inferior a 5 unidades, aplicando clases de CSS dinámicas para pintar las alertas visuales en tonalidades de advertencia.